

Projeto Integrador de Tecnologia da Informação III

Prof. Dr. Hudson Silva Borges

Módulo 2 - Teste de Software Aplicado

Unidade 1 - Conceitos básicos e tipos de testes

Teste de software

- Teste de software é uma técnica de verificação dinâmica.
- Ou seja, visa avaliar o comportamento do programa ou modelo a partir de suas especificações.
- Algumas questões são importantes:
 - O que devemos testar?
 - Como podemos testar?
 - O que esperar dos nossos testes?

Testes: o que devemos testar?

- Sistemas modernos são compostos por dezenas, centenas e até milhares de componentes; devemos testar todos?
- Imagine também que cada componente pode apresentar diversas condições que devem ser avaliadas.
- Seria viável testar todos visando cobrir 100% das situações possíveis? Não!

Testes: o que devemos testar?

- Planejamento é fundamental para que os testes sejam efetivos.
- Devemos estabelecer **objetivos claros**.
- Devemos alinhar os testes ao **perfil da aplicação e seus usuários**.
- Criar **bons testes** (veremos isso adiante).

Testes: como podemos testar?

- Testes **caixa branca**:
 - Conhecem o interior do artefato;
 - São estrategicamente guiados pela estrutura;
 - Encontram erros.

```
37  async inscrever(usuarioId: string, planoId: string, opts?: { duracaoEmMeses: number }): Promise<Assinatura> {
38      const antiga = await this.assinaturaRepo.findOneBy({ usuario: usuarioId, status: 'ativa' });
39      if (antiga) throw new AssinaturaError('Usuário já possui uma assinatura ativa');
40      const plano = await this.planoRepo.findOneBy({ id: planoId });
41      if (!plano) throw new AssinaturaError('Plano não encontrado');
42      const fim = opts?.duracaoEmMeses ? days().add(opts.duracaoEmMeses, 'month').toDate() : undefined;
43      return this.assinaturaRepo
44          .save(new Assinatura({ plano, usuario: usuarioId, fim_em: fim }))
45          .then((assinatura) => this.processar(assinatura.id).then(() => assinatura));
46  }
```

Testes caixa branca tem conhecimento total da estrutura.

Fonte: Elaboração própria.

Testes: como podemos testar?

- Testes caixa preta:
 - Não conhecem o interior do artefato
 - São guiados por requisitos e contratos de interfaces
 - Encontram falhas

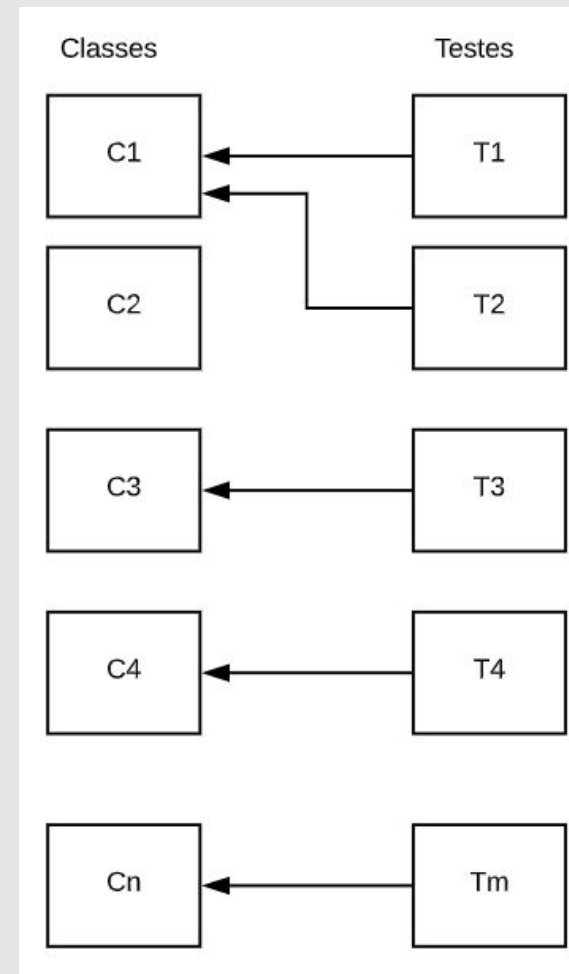
All Methods	Instance Methods	Abstract Methods	Default Methods
Modifier and Type		Method and Description	
boolean		add(E e) Appends the specified element to the end of this list (optional operation)	
void		add(int index, E element) Inserts the specified element at the specified position in this list (optional operation)	
boolean		addAll(Collection<? extends E> c) Appends all of the elements in the specified collection to the end of this list (optional operation)	
boolean		addAll(int index, Collection<? extends E> c) Inserts all of the elements in the specified collection into this list at the specified position (optional operation)	

Documentação da interface *List* do Java.

Fonte: [Documentação JavaSE](#)

Testes: testes unitários

- Testes unitários visam avaliar pequenas unidades;
- Focam no teste de caminhos (caixa branca);
- Feitos pelo próprio desenvolvedor, buscam cobertura de 100% das linhas de código;
- Captura erros na lógica do código.



Testes: testes unitários

- Técnicas comuns para projeto de testes:
 - Classes de equivalência
 - Análise de valor limite
 - Tabela de decisão
 - Transição de estados

#139 - A Developer's Guide to Effective
Software Testing

techleadjournal.dev/episodes/139

> TechLead
| .Journal()
| _



Empirical research shows that bugs love boundaries. Because we are very good at implementing the happy paths, the bugs then start to cluster on things that we're not so good at. That is the corner cases.



Maurício Aniche
Author of "Effective Software Testing"

Técnicas: **Classes de equivalência**

- **Objetivo:**

Separa-se as entradas em classes/grupos com comportamentos semelhantes.

- **Exemplo:**

Verificar maioria de usuários em websites.

Válidos: ≥ 18 anos

Inválidos: < 18 anos

Técnicas: análise de valor limite

- **Objetivo:**

Busca-se avaliar os limites das classes de equivalência.

- **Exemplo:**

Verificar maioria de usuários em *websites*.

Casos de teste: 17, 18, 19 anos

Técnicas: Tabela de decisão

- **Objetivo:**

Em cenários complexos, busca identificar todas as combinações de condições para garantir maior cobertura.

- **Exemplo:**

Algoritmo para avaliação de empréstimos.

Critérios: perfil, histórico, renda, garantias, finalidade, etc.

Como testar todos os cenários? Impossível!?

Técnicas: transição de estados

- **Objetivo:**

Técnica para avaliação de fluxos e estados em aplicações.

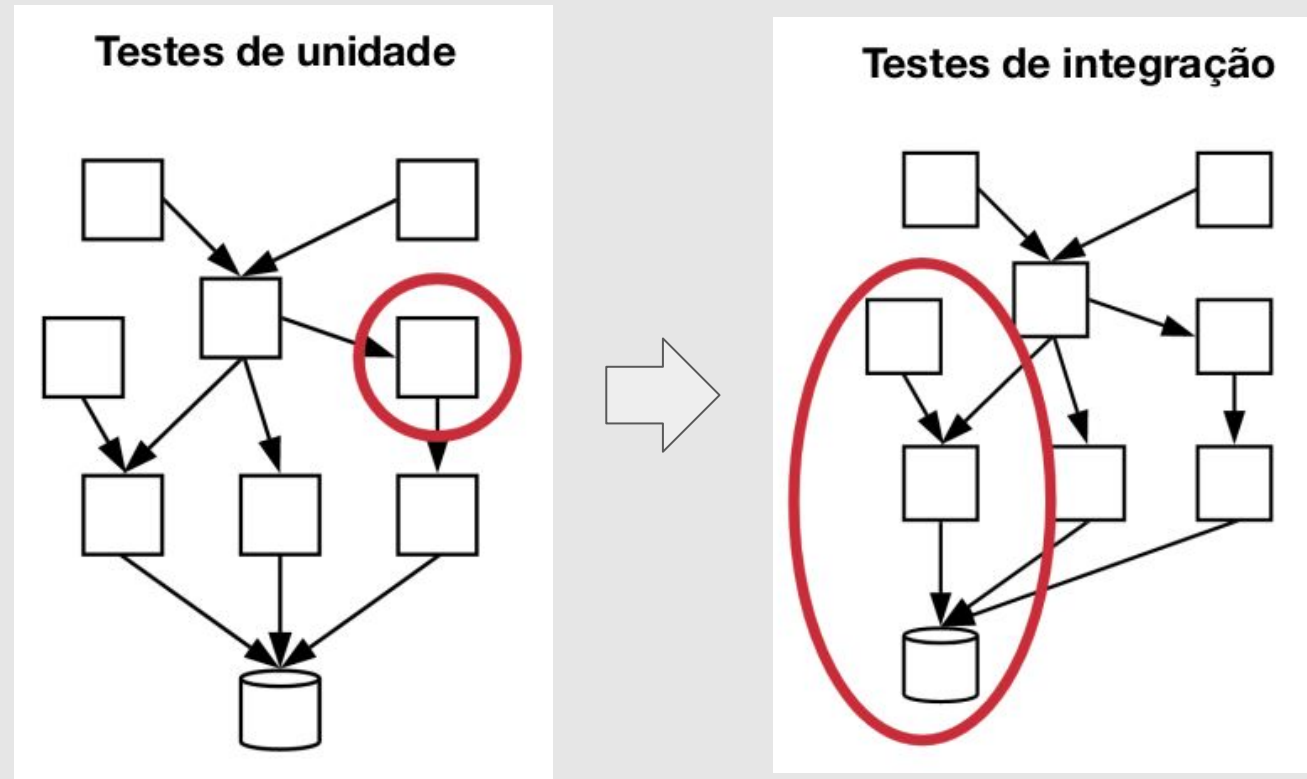
- **Exemplo:**

Lidar com estados de usuários: *deslogado*, *logado*, *bloqueado*, etc.

Testes: testes de integração

- **Verificam a interação** entre componentes ou uma funcionalidade completa do sistema de modo separado da interface de usuário.
- Podem envolver a comunicação com serviços externos, como banco de dados, APIs externas e sistemas de arquivos.
- **Mais lentos, mais caros e menos frequentes** que os testes de unidade.

Testes: testes de integração

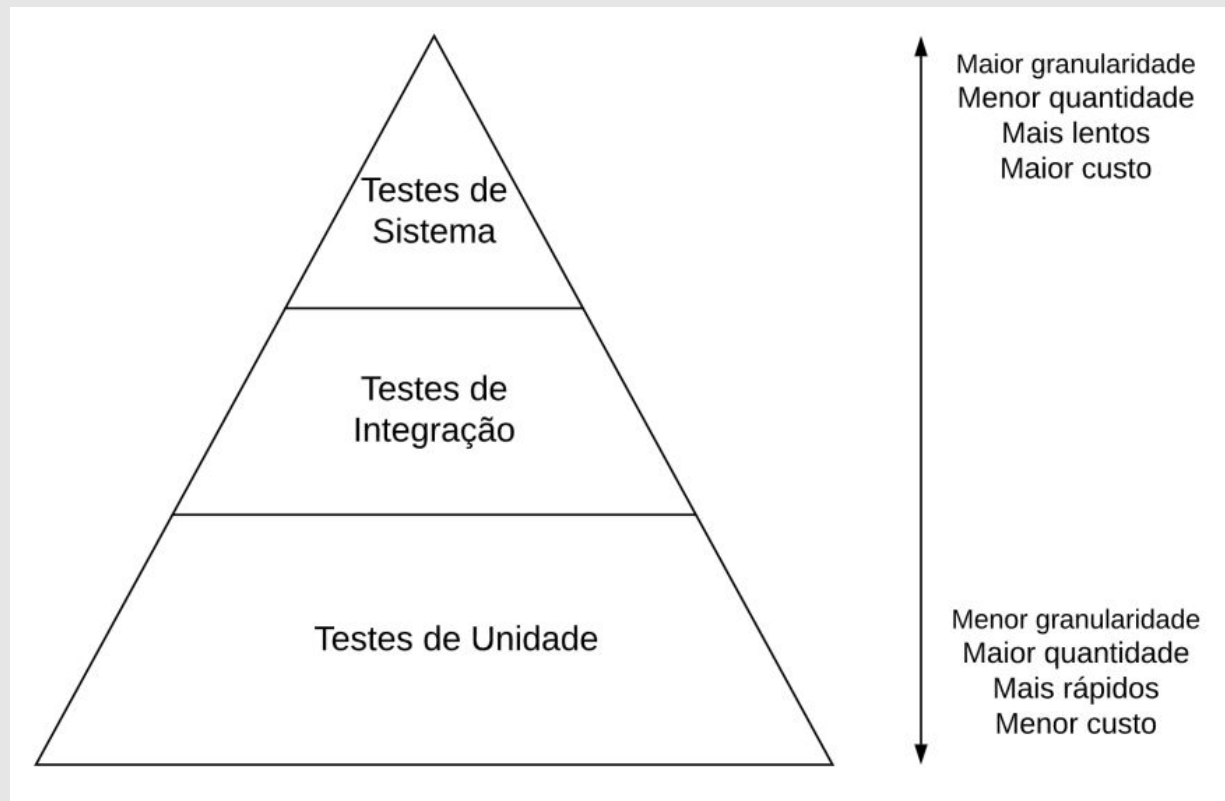


Fonte: Valente, 2020.

Testes: Testes de sistema

- **Simulam o uso do sistema** por usuários e são normalmente realizados na UI do sistema.
- **Mais lentos, mais caros e menos frequentes** quando comparados aos testes de unidade e integração.
- **Mais frágeis**, uma vez que podem ser mais facilmente impactados por alterações na UI.

Testes: testes de sistema



Fonte: Valente, 2020.

Testes: e muito mais ...

- Existem vários outros tipos de testes:
 - Teste de segurança;
 - Teste de carga e stress;
 - Teste de aceitação;
 - e outros.

Módulo 2 - Teste de Software Aplicado

Visão integradora aplicada



UFMS

AGEAD

Agência de Educação
Digital e a Distância

Por onde começar?

- Em projetos já iniciados, como podemos iniciar com testes?
- Depende! Mas sempre devemos considerar as unidades mais críticas que tem potencial de impacto significativo no negócio.
- Neste módulo o objetivo será no **planejamento e aplicação** de testes para verificação dinâmica de componentes.

Próxima unidade



Unidade 2 - Projeto e desenvolvimento de testes

Referências

DELAMARO, Márcio Eduardo; MALDONADO, José Carlos; JINO, Mario. **Introdução ao teste de software**. Rio de Janeiro: Gen Ltc, 2016. ISBN 9788595155732.

GONÇALVES, Priscila de Fátima et al. **Testes de software e gerência de configuração**. Porto Alegre: Sagah, 2019. ISBN 9788595029361.

SOMMERVILLE, Ian. **Engenharia de software**. 9. ed. São Paulo: Pearson, 2011. ISBN: 9788579361081.

VALENTE, Marco Tulio. **Engenharia de software moderna**: princípios e práticas para desenvolvimento de software com produtividade. v.1, n.24, 2020. Disponível em: <https://link.ufms.br/bljNp> . Acesso em 20 dez 2024.

Licenciamento



Respeitadas as formas de citação formal de autores de acordo com as normas da ABNT NBR 6023 (2018), a não ser que esteja indicado de outra forma, todo material desta apresentação está licenciado sob uma [Licença Creative Commons - Atribuição 4.0 Internacional](https://creativecommons.org/licenses/by/4.0/).

